

Review #709A

Overall merit

3. Weak accept

Reviewer expertise

2. Some familiarity

Paper summary

The paper introduces GenScale, an end-to-end cluster orchestration system designed to optimize resource scheduling for genomic workflows without triggering Out-of-Memory (OOM) failures or wasting memory. This task is uniquely challenging because execution times and memory requirements vary significantly across different inputs and hardware vendors. To address this, GenScale integrates three key components: a fine-grained active memory recommender (FAMR) that balances OOM risks and resource waste using per-tool statistics, a cross-platform execution predictor (PTS-hybrid) that utilizes lightweight microbenchmarks, and a cluster sizing optimizer that leverages Bayesian optimization to rapidly evaluate configurations without running the full workflow on a physical cluster. Evaluated across 8,000 genomic task executions, GenScale successfully reduces memory-related retry times by up to 96% and cuts memory waste by 37%.

Summary of strengths and weaknesses

Strengths:

- The paper is highly readable, clearly motivating the core problem and its associated challenges. Specifically, it effectively articulates why uniform memory allocation policies fail, noting their inability to scale against an exponentially growing number of configurations.
- The evaluation is quite comprehensive, suggesting GenScale yields minimal memory wastage and retry across the SOTA baselines, and also the minimal prediction error on the execution time across the vendors.
- The cluster sizing is decided within a reasonable amount of time across 2^n possible subsets using Branch-and-Bound (BnB) approach, where it relies on exploring a search tree and pruning subtrees that yield the worst performance (Refer to equation 5 in the paper)

Weaknesses:

- A notable weakness is the system's heavy reliance on manual hyperparameter tuning, which places a significant burden on the user. Specifically, the safety margin parameter α must be carefully configured to balance memory efficiency against failure avoidance. Similarly, while the weights w_1 , w_2 , and w_3 are constrained to sum to 1, manually determining these values forces users to explicitly quantify the tradeoffs between performance, cost, and resource ratios. This constraint ultimately requires users to decide beforehand how conservative the allocation policy should be for varying cost thresholds.

- As a followup, this configuration overhead becomes particularly problematic given that the evaluation is strictly limited to the GATK Best Practices DNA pipeline. Genomic workloads vary drastically; an RNA-seq, single-cell, or metagenomics pipeline will have completely different CPU-to-memory footprints. Because the parameters are workflow-dependent and do not auto-tune, a user would face a steep burden to completely re-calibrate alpha and the weight distribution whenever the workload changes, calling into question the system's real-world utility across diverse bioinformatics pipelines.

Detailed comments for authors

- The evaluation of GenScale relies on a maximum input size of only 2.75 GB, whereas standard production genomics datasets (like 30x human WGS) regularly exceed 50 GB to 100 GB. This restricted scale undermines the evaluation because genomic tools often exhibit non-linear memory spikes that sub-3 GB files cannot replicate, OOM failures are already rare at this small size (making the 96% retry reduction unrepresentative), and the system's profiling overhead might outweigh its benefits for such small workloads. The authors should evaluate GenScale on a production-scale dataset to prove its real-world utility, or at minimum, explicitly add this input size constraint to the text as a major system limitation.
- The paper explains how GenScale tries to avoid Out-of-Memory (OOM) crashes, but it does not say what happens when a prediction is wrong and a crash still occurs. If a tool unexpectedly runs out of memory mid-run, how does GenScale recover? Does it automatically move the job to a bigger machine, or does it just default to the baseline behavior of failing the job and requiring a full restart?

[Review #709B](#)

Overall merit

2. Weak reject

Reviewer expertise

2. Some familiarity

Paper summary

GenScale sizes heterogeneous clusters for genomic workflows using three components -- FAMR (distribution-based memory recommender), PTS-Hybrid (microbenchmark-driven execution-time predictor), and a BnB+BO optimizer over a workflow simulator. It is evaluated on 8,000 task executions on Chameleon Cloud with bare-metal validation on 9 configurations.

Summary of strengths and weaknesses

The problem is important and well motivated, and the bare-metal validation is strong. But the three components run in sequence with no coupling, so the integrated framing overstates a combination of known techniques. The memory baseline ignores retry-based memory escalation used in production, and PTS-Hybrid loses to Lotaru on the two content-driven tools, one of which anchors the entire memory-variability motivation. Cross-platform evaluation covers only three x86 CPUs, so the generalization claim is narrow.

Detailed comments for authors

This paper presents GenScale, a system for sizing heterogeneous clusters for genomic workflows under cost and deadline constraints. It has three components: FAMR, a memory recommender that models per-tool memory as distributions and weights allocation by peak timing; PTS-Hybrid, an execution-time predictor that maps microbenchmark scores to per-tool processing rates via ridge regression, and an optimizer that combines branch-and-bound with Bayesian optimization over a simulator to select a machine subset. Evaluation uses 8,000 task executions on Chameleon Cloud, with bare-metal validation on 9 configurations. The motivation is well supported.

The framing oversells what the system does. The intro claims three coupled decisions solved jointly, but the components run in sequence with no feedback. FAMR emits a memory number, PTS-Hybrid emits a time, the optimizer consumes both. The one place coupling matters is memory headroom versus machine selection, and that is where it is missing. FAMR's safety margin is fixed before the optimizer runs, but the alpha parameter trades directly against cost through machine packing. Make it or the target percentile a decision variable inside the optimizer. As written, integrated describes a decoupled pipeline of three known techniques.

Another important gap is a missing baseline tied to an unmodeled core problem. Production engines like Nextflow and nf-core already handle OOM by escalating memory on retry, multiplying allocation per attempt. That attacks the same trade-off FAMR targets. The retry-time metric assumes naive full-restart, so the FAMR advantage may shrink against escalation and you need to show it does not. Also, the real driver is content-driven variability from read depth and duplication, which Finding 1 names but no component models. FAMR absorbs it by widening allocation. PTS-Hybrid increases the error and trails Lotaru on exactly PMd and PCWM, the two content-driven tools, with PMd also being the tool the entire memory-variability story rests on. FastQC runs first in the pipeline and already reports duplication levels. That is a cheap early signal available before PMd runs. Feeding it downstream converts absorbed noise into a predictable feature and gives the paper a modeling insight.

The cross-platform evaluation is narrower than the claim requires. The three machines are all x86 server CPUs, and the 30 machine types are VM partitions of those three, so the predictor effectively sees three architecture points. Adding wider spread such as ARM or Graviton would test whether the benchmark-to-rate mapping holds across different hardware. Separately, the optimizer is compared only against the authors' pure BO. The 5.5% cost gain needs a stronger baseline, either

OPTIMUSCLOUD adapted here or a reason it does not apply. The variance reduction and faster convergence are stronger and should be foregrounded.

Questions for authors' response

Please refer to the above comments.

Review #709C

Overall merit

3. Weak accept

Reviewer expertise

2. Some familiarity

Paper summary

GenScale is a system for predicting cluster size for genomic workflow computations. The authors propose two new tools for predicting resource consumption and execution time, and design a simulation-based approach for finding the optimal cluster size. GenScale is evaluated on a single genomic workflow with multiple input sizes, demonstrating significant improvement in task time estimation and modest cost improvements.

Summary of strengths and weaknesses

Strengths

- + Interesting and relevant problem.
- + Significant improvements in prediction of execution time and memory consumption.

Weaknesses

- Modest cost improvements for such a complex system.
- Evaluation uses only a single application and could use better baselines.
- Generalizability might be overclaimed.

Detailed comments for authors

Thank you for submitting your work to EuroSys'27. I find the work interesting and targeting an important problem. However, I find the evaluation to be not convincing due to its reliance on a single application and unclear selection of baselines for different experiments. Furthermore, I'm not convinced by Section 8 that the system will generalize to other classes of scientific workloads.

Evaluation

- The evaluation uses a single application only, which does not convince me that the whole approach generalizes and is not overfitted to a single application. In particular, this workload has a long critical path with few parallel paths, which exacerbates the cost of retries.
- Figure 9 shows that in addition to FAMR, the best-performing methods are WittPer and P95, not Sizey. Then why does Figure 10 compare FAMR against Sizey (arguably the worst baseline) and not the best ones? Conducting additional experiments with a limited selection of baselines after performing an overall assessment is an acceptable practice, but only if you exclude worst-performing baselines, not best-performing ones.
- How would the Bayesian Optimization work with existing, best-performing tools: Lotaru-G for time prediction and P95 or WittPer for memory prediction? The paper evaluates the simulator accuracy with Sizey + Lotaru-G (again, not with P95/WittPer), but not the efficiency of the whole pipeline. This seems to be the most interesting experiment, as it would establish if introducing two new prediction tools is even necessary.
- What is the overall cost of the whole system, including full profiling of a single workflow, microbenchmarks, and testing each tool on a sample of input?

Generalizability

In Section 8, the paper claims that the pipeline can be applied to climate simulations. However, the system assumes the ability to execute tools with a 5% input sub-sample (line 407). How will you sub-sample an atmospheric grid? Or is it rather the case that the sub-sampling works for genomics because the operations are data parallel, and executing a 5% subset provides meaningful information that can be later extrapolated?

Furthermore, doesn't FARM assume that the memory allocation changes linearly with the input size (499-502)? I'm also unsure if this assumption will hold for other scientific applications.

Other Issues

- Section 4 seems to be underspecified in some places - for example, how is E_tool obtained?
- In Table 2, what is the interpretation of the total runtime sum of independent tools?
- I don't think this sentence is mathematically correct: "BWA-MEM runs with consistent execution times across AMD EPYC and two Intel Xeon processors, with overlapping confidence bands across all input sizes.". Overlapping confidence intervals do not imply that the runtimes are statistically indistinguishable.
- I'm not sure I can understand lines 1145-1146 in connection with Figure 11. Doesn't Figure 11b show that there's already at least one sample for BO-Classic around \$115? So how can the probability be zero in the text? And for BnB+BO, the figure seems to show the cumulated probability of 25% for \$120, while the text states it's 60%. Furthermore, Figure 11a shows quite clearly that 60% of trials of BnB+BO cannot be below \$120, as the median is above \$120.
- Sentence at 604 is missing a dot at the end.

Review #709D

Overall merit

2. Weak reject

Reviewer expertise

2. Some familiarity

Paper summary

The paper looks at resource provisioning for genomic (bio-informatics) workflows, on heterogeneous cloud VMs. It builds memory and completion-time predictors for tasks, and provisions VMs to reduce risk of out-of-memory conditions. The system GenScale, does profile-guided end-to-end optimization. Heuristics for resource-selection optimizations (branch-and-bound + bayesian optimization) are developed. On the workload evaluated, the predictions are fairly robust, and reduce cost by 5% compared to other systems like Lotaru.

Summary of strengths and weaknesses

Strength

- + Fairly practical system with minimum, effective profiling, and provides full cluster-sizing and selection optimizations.

Weakness

- Heavily reliant on time/memory predictions, which are evaluated for only one workload
- The cost saving is minimal (5%)
- Out-of-memory mitigation is the key contribution. However, swapping or checkpointing are not used/compared.

Detailed comments for authors

**** Motivation/Novelty**

- The paper's key contribution and the foundation of the system is the memory/time prediction. However this is evaluated for only one workflow, which is a severe limitation of the empirical evaluation.
- For out of memory errors, why is swapping or checkpointing not used?
- An important finding is the non-linearity of memory usage with input size. However in section 4.1, (line 500), linear interpolation/extrapolation is used.

****** The paper develops multiple simple optimizations for workflow execution:

- The machine characterization is done using phoronix test suite, which is then correlated with the application performance. Seems reasonable.
- Memory allocation considers the time-wasted
- A job/resource allocation simulator using the task performance models
- Heterogeneous cluster selection using the estimated time and cost functions, using an optimization approach. For reducing the search-space, branch-and-bound and bayesian optimization are used.

** Evaluation

- The focus is largely on the task resource predictions (sections 7.2 and 7.5 since the simulator evaluation is really testing task predictions). However, what happens when the predictions are wrong? A deeper sensitivity study would be good.
- Cluster-level performance analysis seems to be missing.
- Line 879 talks about "9 evaluation configurations", used throughout the evaluation, but not clear what they are (earlier reference to 30 machine configurations in the paragraph).
- For the cluster-sizing optimization, comparing against the true optimum is missing.
- How much profiling cost/time is required?